

SMART CAMPUS PARKING SYSTEM

ParkIQ — a next-generation, QR-based parking intelligence platform for campuses

Complete modular build roadmap · standout S.M.A.R.T. architecture · database & API design
phased 6-week plan · testing checklist · 2-minute demo script · viva preparation

SOLVES: live slot visibility · time-window reservations · QR check-in / check-out



■ AVAILABLE ■ RESERVED ■ OCCUPIED □ COMING SOON — FORECAST

THE S.M.A.R.T. STANDOUT MODEL

S
Sensing mesh

M
Machine learning

A
Assured entry

R
Realtime twin

T
Timetable-aware

Team: _____

Guide: _____

Institution: _____

Date: _____

“Don't just book parking — predict it.”

★

What's Inside This Blueprint

Thirteen sections that take you from problem statement to a deployed, demo-ready system — plus the standout architecture model that separates your team from every other submission.

#	Section	What it covers
1	Problem & Objectives	Actors, goals, and a traceability table mapping every word of the problem statement to a module
2	The Golden Flow	The 8-step user journey every module hangs on
3	The ParkIQ Standout Model	S.M.A.R.T. architecture, 8 unique differentiators, us-vs-them comparison + architecture diagram
4	Digital Twin Live Map	The visual centrepiece of the demo — live campus map
5	Modular Build Roadmap	9 modules with features, build order map and deliverables
6	Database Design	10 core entities, relations, and the overlap-prevention constraint
7	API Endpoints	16 REST routes covering every action in the system
8	Tech Stack	Recommended stack + alternatives, chosen for speed of building
9	QR Security Deep-Dive	Signed single-use tokens, all rejection cases, sequence diagram
10	Phased Build Plan	6 weeks, 4 milestones, who-does-what parallelisation
11	Testing Checklist	10 must-pass scenarios including the two-browser double-booking test
12	Demo Script & Viva Prep	The 2-minute demo that wins + 8 questions evaluators will ask
13	Future Scope	The roadmap after v1 — for the presentation's final slide

1

Problem Statement & Objectives

THE PROBLEM STATEMENT

“Build a smart campus parking system that allows students and staff to view available parking slots, reserve a slot for a specific time, and use a QR-based system for parking check-in and check-out.”

Objectives — what “done” looks like

- Real-time visibility** — any user sees which slots are free right now, per lot, per vehicle type, for a chosen time window.
- Guaranteed reservations** — a slot booked for a window belongs to exactly one user; double-booking is impossible, no-shows auto-expire.
- Frictionless QR entry/exit** — one signed, single-use QR starts and ends the parking session; forged or stale codes are rejected.
- Fair utilisation** — overstay detection, admin overrides and off-peak nudges keep slots turning over.
- Actionable analytics** — occupancy, peak hours, no-show rates and a sustainability meter for campus administrators.
- Software-first** — the whole system works end-to-end without any hardware; IoT sensors and a servo gate are optional demo upgrades.

Who uses the system

Actor	What they do
Student / Staff	View live availability, reserve a slot for a time window, receive & show a QR, check in and out
Admin (security office)	Manage lots/zones/slots, see live occupancy, handle overstays & violations, view reports
Security / Gate kiosk	Scan QR at entry and exit, see currently-parked list, force-end sessions
System (automated)	Expire no-shows, free slots, arm overstay timers, send notifications, update the live map

Requirement traceability — every word of the problem, covered

Problem requirement	Where ParkIQ meets it
“students and staff” use the system	Module 1 — role-based accounts (STUDENT / STAFF / ADMIN) verified by college email
“view available parking slots”	Module 3 — live availability engine + Digital Twin map, pushed over WebSocket in < 1 s
“reserve a slot for a specific time”	Module 4 — time-window reservation engine with DB-level overlap prevention and no-show auto-expiry
“QR-based check-in and check-out”	Module 5 — signed, single-use, time-bound QR verified at the gate; geofence auto check-out as a bonus layer

NOTHING LEFT UNIMPLEMENTED

Every clause of the problem statement maps to a named module and a test in Section 11. When the evaluator asks “does it meet the problem statement?” — this table is your answer.

2 The Golden Flow — One Journey to Rule the Build

The entire project is one user journey. Every module you build is a piece of this flow — print it and stick it on the team wall.

1. **Login** — student or staff; role-based dashboard (Module 1)
2. **View live availability** — digital twin map + counts, filtered by lot, vehicle type and time window (Module 3)
3. **Reserve a slot** — pick a time window; ClassSync suggests the smartest lot; booking is overlap-safe (Module 4)
4. **Receive the QR pass** — signed, single-use, time-bound — lands in the app and by email in seconds (Modules 5 + 7)
5. **Check in at the gate** — scan the QR; the edge kiosk verifies it; barrier opens; the slot flips red on the live map (Module 5)
6. **Park** — the slot sensor confirms presence; overstay timer arms itself (Modules 6 + 9)
7. **Check out** — scan again — or geofence auto-checkout fires when you leave campus; receipt with duration & CO₂ saved (Modules 5 + 6)
8. **Slot freed** — availability updates everywhere in under a second — the loop closes (Module 3)

3 The ParkIQ Standout Model — S.M.A.R.T.

Most teams will submit the same thing: a CRUD app with a QR image. It works — and it looks like every other submission. ParkIQ keeps that rock-solid core and wraps it in a layer of upcoming technology that no other team will demonstrate: AI forecasting, a realtime digital twin, edge computing and geofencing. Same problem statement; a generation-ahead answer.

S.M.A.R.T. — the five pillars

Pillar	What it means
S Sensing mesh	Per-slot IoT sensors and an optional ANPR camera feed the twin with ground truth, not guesses
M Machine learning	Occupancy forecasting and the timetable-aware slot allocator
A Assured entry	Signed single-use QR verified at the edge; hybrid QR / ANPR / RFID identity
R Realtime digital twin	A live map of every slot on campus — sub-second updates and historical replay
T Timetable-aware	Bookings that understand class schedules and suggest the smartest slot & duration

The 8 differentiators (each is a demo moment)

1 • ParkIQ Forecast — AI occupancy prediction [SOFTWARE · HIGH IMPACT]

TECH: Prophet (or a small LSTM) trained on booking + session history; features: hour, weekday, exam calendar, campus events

Predicts each lot's occupancy for the next 48 hours and renders it as a heatmap. Students see “P1 will be full by 9:40 — P3 has space till 11:15” and can book before they even reach campus. **Why it stands out:** no other team will show a system that knows the future of your parking.

2 • ClassSync Allocator — timetable-aware booking [SOFTWARE · HIGH IMPACT]

TECH: timetable CSV import + block-to-lot distance matrix

When a student books at 9:50 for a 10:00 class in Block C, ParkIQ proposes the nearest lot with real availability and a duration of class-end + 15 minutes — one tap to accept. **Why it stands out:** campus-native personalisation that no generic parking app has.

3 • Digital Twin Live Map [SOFTWARE · THE DEMO MOMENT]

TECH: SVG / Leaflet campus map + WebSocket push

Every slot is a live coloured cell (green / amber / red) with per-lot counts, a “You → B-07” pin to your reserved slot, and admin replay (“show me Tuesday 9 AM”). **Why it stands out:** evaluators remember what they can see — this is the centrepiece (Section 4).

4 • EdgeGuard Gates — offline-capable kiosks [HARDWARE · VIVA GOLD]

TECH: Raspberry Pi kiosk; JWT signature (HMAC) verified locally; cached revocation list; offline event queue (Rs 3–4k)

The gate kiosk verifies QR signatures locally, so check-in keeps working when campus WiFi dies; events sync to the cloud when the link returns. **Why it stands out:** you answer “what if the internet fails?” before the evaluator asks it.

5 • Triple-ID entry [HARDWARE · OPTIONAL]

TECH: QR (primary) · ANPR camera express lane (OpenALPR) · RFID card fallback

Redundant identity at the gate: phone dead? swipe your RFID card. Staff cars get the camera lane. **Why it stands out:** a reliability story with no single point of failure.

6 • Geofence auto check-out [SOFTWARE · UNIQUE]

TECH: PWA geolocation + haversine distance to the campus boundary

If a user leaves campus with a session still open, ParkIQ waits a 5-minute grace period, closes the session automatically and emails the receipt. No slot is ever stuck “occupied” by a forgetful user. **Why it stands out:** a closing move almost no student team implements.

7 • GreenMeter — sustainability analytics [SOFTWARE · EASY WIN]

TECH: *simple arithmetic on real data: search-minutes avoided × average fuel burn*

A live dashboard counter — “this month: 96 hours of searching saved · 214 kg CO₂ avoided” — turns a parking project into a green-campus story. **Why it stands out:** administrators love it, and evaluators quote it back to you.

8 · FairFlow — off-peak rewards [SOFTWARE · NICE-TO-HAVE]

TECH: *points ledger in the database + a small rules engine*

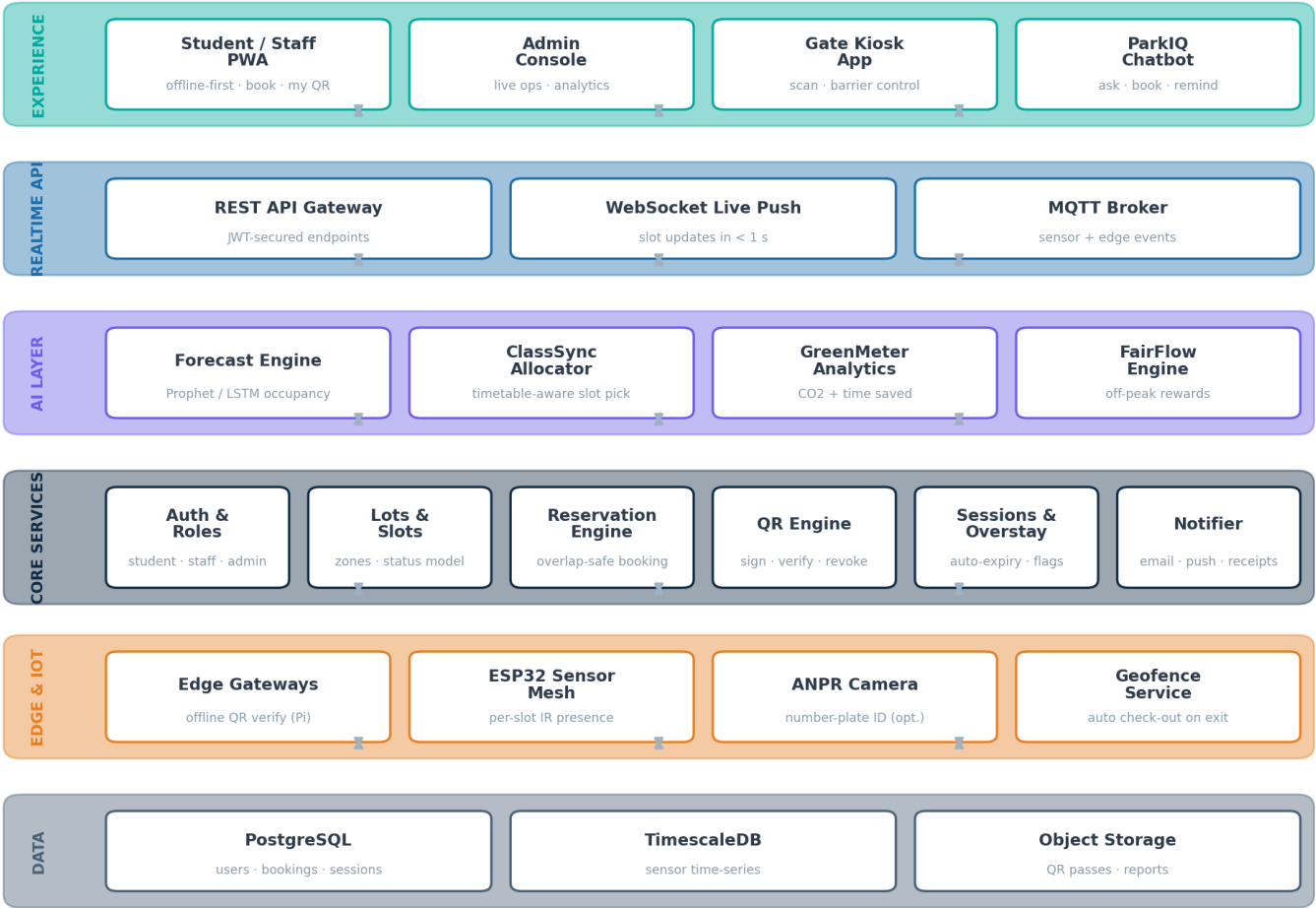
Parking outside the 9 AM crush earns FairFlow points, gently shifting demand across the day — without any payment gateway or money handling. **Why it stands out:** behavioural engineering with zero payment complexity.

Us vs. the typical team — the moat table

Dimension	Typical team project	ParkIQ (yours)
Availability	manual page refresh	WebSocket push < 1 s + 48-hour ML forecast heatmap
Slot choice	user picks any free slot	ClassSync suggests the lot nearest their next class + duration
Entry tech	a QR image	signed single-use QR + optional ANPR + RFID fallback
Gate logic	cloud-only — dies offline	EdgeGuard kiosk verifies offline, syncs later
Check-out	manual scan only	scan + geofence auto-checkout with grace period
No-show handling	slot stays blocked	20-minute auto-expiry frees the slot and notifies
Data story	static tables	Digital Twin live map + GreenMeter CO ₂ analytics
Wow factor	a CRUD demo	“book before you arrive” AI + live twin on the projector

ParkIQ — Next-Gen Layered Architecture

The S.M.A.R.T. model · Sensing mesh · Machine-learning intelligence · Assured entry · Realtime digital twin · Timetable-aware



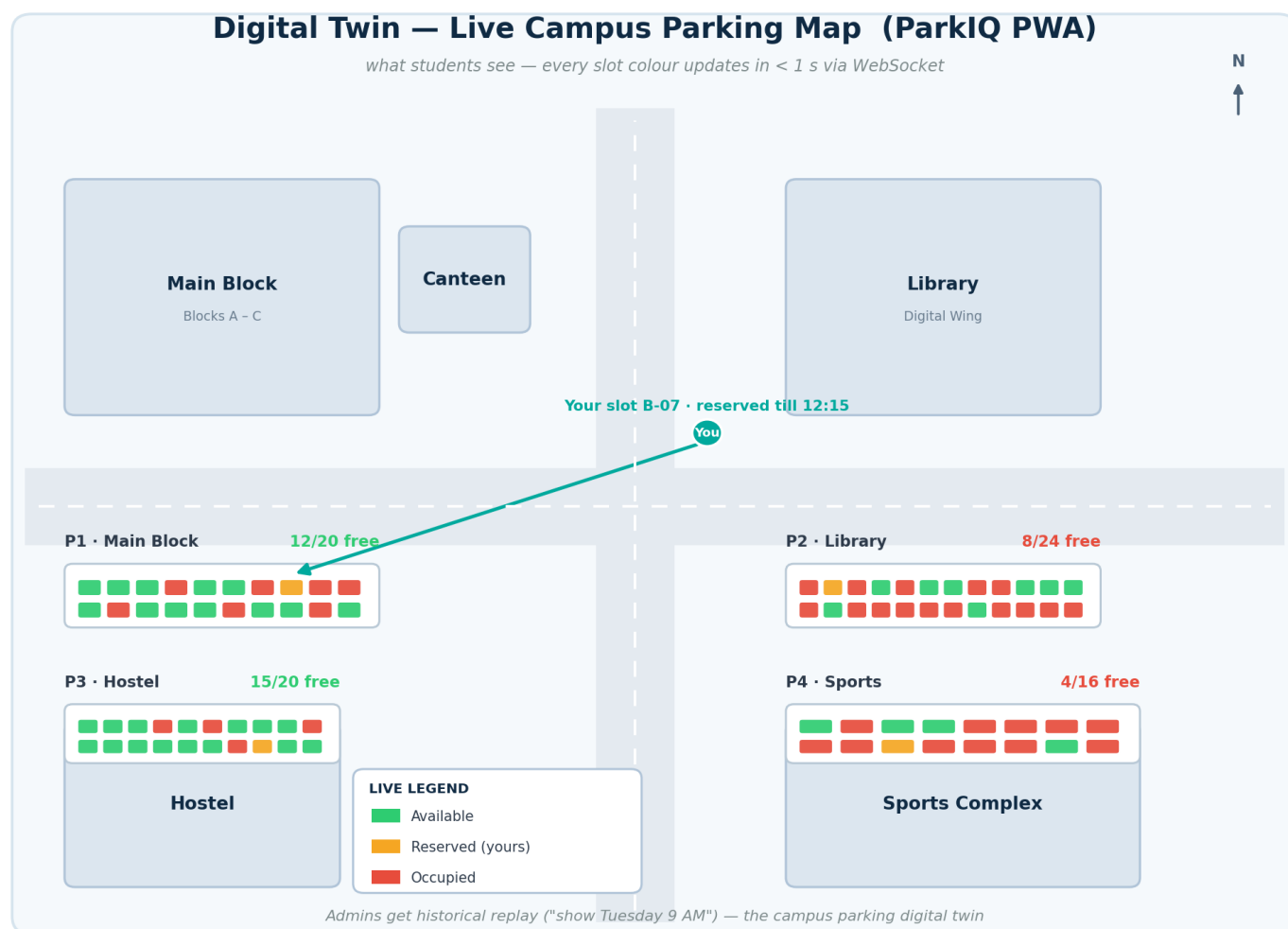
All traffic TLS + JWT-authenticated · every action audit-logged · edge devices survive network outages

Figure 1 — ParkIQ next-gen layered architecture. Solid core (Modules 1–6) wrapped in AI, edge and realtime layers.

FEASIBILITY REALITY CHECK

Differentiators 1, 2, 3, 6 and 7 are pure software — they fit inside the 6-week plan below. Differentiator 4 (EdgeGuard) is a small Raspberry Pi demo if budget allows. Differentiator 5 (ANPR) can be shown as a recorded video or mock screen. Never let the wow layer break the core: a working QR loop beats a broken AI demo every time.

4 Digital Twin Live Map — The Visual Centrepiece



What it is — the campus drawn as an interactive map where every physical slot has a live digital cell: green available, amber reserved (yours), red occupied.

How it's built — an SVG or Leaflet map layer; slot cells bound to a WebSocket channel; one state change = one pushed update, no refresh.

The “You” pin — after booking, the map drops a pin and a walking path to your exact slot — tiny feature, huge demo effect.

Historical replay (admin) — scrub through yesterday to answer “when does P1 actually fill?” — this data is also what trains ParkIQ Forecast.

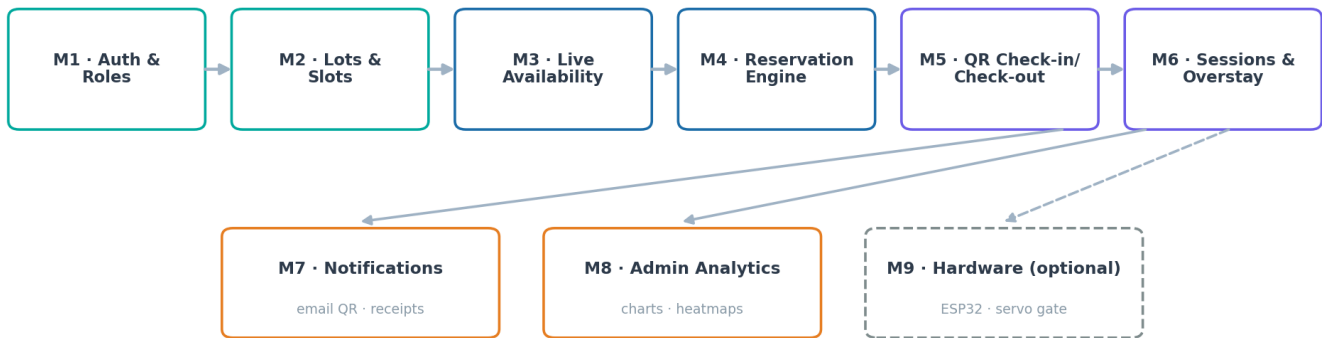
Why evaluators love it — it is the one screen that makes the whole system instantly understandable to a non-technical audience.

DEMO MOMENT

Book a slot live on the projector and watch the cell flip green → amber in front of the evaluator's eyes. Then scan the QR on a second phone and watch it flip amber → red. Thirty seconds, unforgettable.

5 Modular Build Roadmap — 9 Modules, 9 Deliverables

Module Map — Build Order & Dependencies



M1 → M6 is the critical path (Phases 1-3) · M7 / M8 run in parallel with Phase 4 · M9 is the optional hardware sprint (Phase 5)

Figure 3 — module map. M1→M6 is the critical path; M7/M8 run in parallel; M9 is the optional hardware sprint.

Module 1 · User & Role Management (Phase 1 · Week 1)

Register / login with college email (roll no. / employee ID), bcrypt password hashing

Roles: STUDENT, STAFF, ADMIN — JWT sessions on every request

Role middleware guards admin-only routes

Deliverable: three role-based dashboards behind secure login.

Module 2 · Parking Lot & Slot Management (Phase 1 · Week 1)

Admin CRUD for lots → zones → slots (codes like LIB-A-07)

Slot types: 2-wheeler / 4-wheeler / EV / accessible

Status model: AVAILABLE · RESERVED · OCCUPIED · MAINTENANCE

Deliverable: admin creates a 50-slot lot; the grid shows every slot's status.

Module 3 · Live Availability Engine (Phase 2 · Week 2)

The availability query: slot free AND no overlapping active reservation

Filters by lot, vehicle type and requested time window

Live counts per lot + WebSocket push on every state change

Deliverable: the live grid updates in under a second after any booking or checkout.

Module 4 · Reservation & Booking Engine (Phase 2 · Week 2-3)

Book a slot for a time window — auto-assign or hand-picked

Rules: no overlap (DB transaction + row lock), max 4 h, one active booking, today/tomorrow only

States: PENDING → CONFIRMED → CHECKED_IN → COMPLETED (+ EXPIRED / CANCELLED)

No-show job: auto-expire 20 min after start time and free the slot

Deliverable: two browsers booking the same overlapping slot — exactly one succeeds.

Module 5 · QR Check-In / Check-Out (Phase 3 · Week 3)

QR carries a signed JWT: reservation ID, user, slot, expiry

Gate kiosk / scanner phone calls POST /checkin and /checkout

Single-use state, replay protection, time-window enforcement

QR pass lands in-app and by email (PDF) instantly

Deliverable: the full loop: book → QR → scan → park → scan → freed, with receipts.

Module 6 · Session & Time Management (Phase 3 · Week 4)

- parking_sessions: check-in/out times, duration, slot, user
- Live “currently parked” list for security staff
- Overstay detection → flag + admin alert; admin force-end for forgotten sessions
- Deliverable:** admin sees “42 vehicles parked now” plus overstay alerts and full history.

Module 7 · Notifications (Phase 4 · Week 4-5)

- Booking confirmation with QR attached · 15-min reminder before start
- Check-in / check-out receipts with duration and CO₂ saved
- Expiry, overstay and admin alerts — all as an event queue, cleanly separated
- Deliverable:** the QR email arrives within seconds of booking — demo it live.

Module 8 · Admin Dashboard & Analytics (Phase 4 · Week 5)

- Live occupancy gauges, peak-hour heatmap, utilisation %, no-show rate
- GreenMeter sustainability counters and FairFlow leaderboards
- CSV export for campus administration reporting
- Deliverable:** a dashboard an evaluator will photograph.

Module 9 · Hardware (Optional Sprint) (Phase 5 · Week 5)

- ESP32 + IR sensor per slot for 2-3 demo slots (POST /slots/:id/status)
- Servo barrier that opens on successful check-in
- Raspberry Pi EdgeGuard kiosk for offline-capable scanning
- Deliverable:** a physical gate that opens on a scan — demo-day magic, never a dependency.

6 Database Design

Core entities

Entity	Key fields	Purpose
users	id, name, email, password_hash, role, vehicle_no, roll_no	Accounts & roles (M1)
parking_lots	id, name, description, 2W/4W capacity	Physical lots (M2)
parking_slots	id, lot_id, zone, slot_code, type, status	Every individual slot (M2, M3)
reservations	id, user_id, slot_id, start_time, end_time, status, vehicle_no	Time-window bookings (M4)
parking_sessions	id, reservation_id, user_id, slot_id, check_in_at, check_out_at, status, overstayed	Physical presence & duration (M6)
qr_tokens	id, reservation_id, token, issued_at, used_at	QR audit trail + single-use state (M5)
notifications	id, user_id, type, message, sent_at, read	Outbox / history (M7)
timetables	id, user_id, day, class_block, start_time, end_time	Class schedules for ClassSync (M4)
green_stats / fairflow_points	id, user_id, session_id, minutes_saved, points	GreenMeter & rewards (M8)
audit_logs	id, actor, action, entity, timestamp	Who did what — every action (security)

The constraint that kills double-booking

Overlapping active reservations for one slot must be impossible at the database level — not just in application code. PostgreSQL can enforce it directly:

```
-- PostgreSQL: two ACTIVE reservations for one slot may never overlap in time
ALTER TABLE reservations ADD CONSTRAINT no_overlap
EXCLUDE USING gist (
  slot_id WITH =,
  tstzrange(start_time, end_time) WITH &&
) WHERE (status IN ('CONFIRMED', 'CHECKED_IN'));
```

On MySQL, enforce the same rule inside a transaction: SELECT ... FOR UPDATE on the slot row, re-check for overlaps, then INSERT — all inside BEGIN/COMMIT.

7

API Endpoints — 16 Routes That Cover Everything

Method	Endpoint	Who	Purpose
POST	/api/auth/register	public	Sign up with college email
POST	/api/auth/login	public	Login → JWT
GET	/api/lots	user	List lots + live availability counts
GET	/api/lots/{id}/slots?from&to&type	user	Slots free for a time window
GET	/api/twin/live	user	WebSocket-ready live map state
GET	/api/forecast?lot&hours	user	48-hour occupancy prediction (AI)
POST	/api/reservations	user	Book a slot (overlap-safe)
GET	/api/reservations/mine	user	My bookings + QR passes
PATCH	/api/reservations/{id}/cancel	user	Cancel a booking
POST	/api/checkin	kiosk/admin	Verify QR → start session, open barrier
POST	/api/checkout	kiosk/admin	End session → free slot
POST	/api/slots/{id}/status	IoT sensor	ESP32 presence update
POST	/api/geofence/exit	user (auto)	Auto check-out on leaving campus
GET	/api/admin/occupancy	admin	Live occupancy stats
GET	/api/admin/analytics	admin	Heatmaps, GreenMeter, CSV export
POST	/api/admin/lots /slots	admin	Manage lots, zones, slots

8

Recommended Tech Stack

Layer	Recommended	Alternatives
Frontend	React + Tailwind CSS (PWA, responsive web app)	Vue, Flutter mobile app
Backend	Python FastAPI (or Node.js + Express)	Django, Spring Boot
Database	PostgreSQL (+ TimescaleDB for sensor data)	MySQL, MongoDB
QR generation	qrcode (pip) / qrcode.react (npm)	segno
QR scanning	html5-qrcode (browser camera)	pyzbar, zxing
Auth	JWT (pyjwt / jsonwebtoken) + bcrypt	session cookies
Realtime	WebSocket (Socket.IO / FastAPI websockets)	Server-Sent Events
ML forecasting	Prophet (fast, robust) → LSTM (stretch)	scikit-learn regression
Charts / map	Recharts or Chart.js + Leaflet/SVG map	Plotly, Mapbox
Email	Nodemailer / smtplib / Brevo free tier	Resend, SendGrid
IoT (optional)	ESP32 + IR sensor + MQTT + servo	Raspberry Pi, LoRa
Deploy	Render / Railway + Neon / Supabase (free tiers)	AWS EC2 + RDS

TEAM RULE

Choose the stack at least one teammate already knows. FastAPI + React + PostgreSQL is the strongest resume-worthy combo; a team on a familiar stack ships twice as fast as a team on a fashionable one.

9 QR Security Deep-Dive — Your Viva Gold

How the QR actually works

- 1. Sign, don't just encode** — on booking, the backend embeds a payload in a JWT signed with a server secret. A raw booking ID in a QR is forgeable — a signature is not.
- 2. Ship it twice** — the QR appears in the app instantly and arrives as a PDF pass by email — phones die; inboxes don't.
- 3. Verify at the edge** — the gate kiosk checks the HMAC signature and time window locally — offline-capable — then confirms single-use state with the server.
- 4. Flip the slot** — on success: session created, slot OCCUPIED, barrier opens, live map pushes the change.
- 5. Close the loop** — exit scan (or geofence) ends the session, frees the slot and emails the receipt with duration and CO₂ saved.

What's inside the QR (and what it proves)

```
{  
    # JWT payload — signed, not encrypted  
    "reservationId": "RES-1024", # which booking this proves  
    "userId": "u_88",           # whose booking  
    "slotId": "LIB-A-07",       # where they may park  
    "exp": 1798760000           # when the proof expires  
}  
SIG = HMAC-SHA256(payload, SERVER_SECRET) # verified at gate + cloud
```

Every rejection case — know these cold

Forged QR — signature check fails; the kiosk rejects before touching the database.

Expired QR — past the exp claim; yesterday's screenshot is dead on arrival.

Replayed QR — single-use state in qr_tokens; the second scan of a used check-in token is refused.

Outside the window — a valid QR scanned hours before the booked start is politely declined (+15-min grace).

Cancelled / expired booking — the reservation state is re-checked server-side at scan time.

Wrong context — the kiosk sends its own signed identity; a rogue scanner cannot inject sessions.

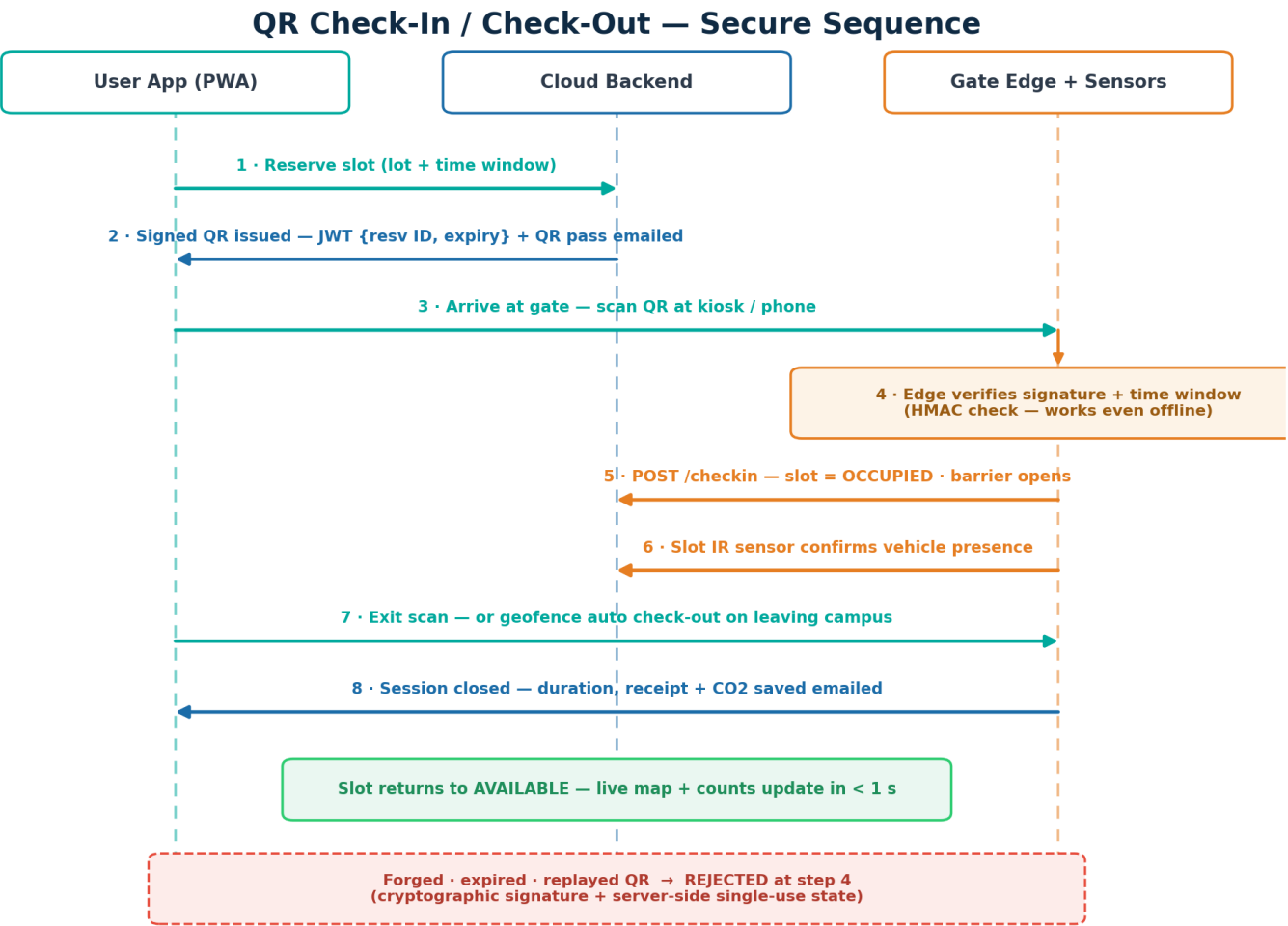


Figure 4 — the full check-in / check-out sequence, including the offline verification path and every rejection.

10

Phased Build Plan — 6 Weeks to Demo Day

Phased Build Plan — 6 Weeks to Demo Day

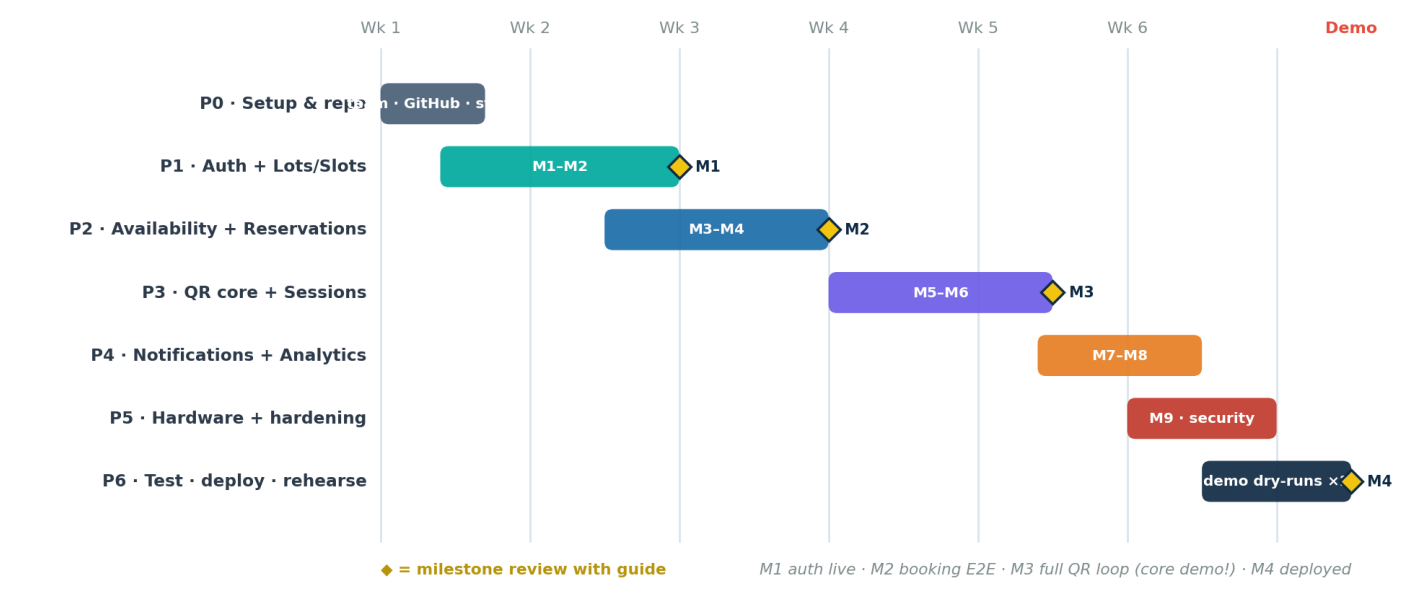


Figure 5 — the 6-week plan. Diamonds are milestone reviews with your guide

Phase	Weeks	Modules	Milestone / exit criteria
P0 · Setup	0–0.7	—	Repo, branches, stack running, blank deploy live
P1 · Foundation	0.4–2	M1–M2	M1: auth works for 3 roles; admin can build a lot
P2 · Availability & Booking	1.5–3	M3–M4	M2: end-to-end booking with zero double-booking
P3 · QR Core	3–4.5	M5–M6	M3: the full QR loop — core demo works!
P4 · Polish	4.4–5.5	M7–M8	Emails out, dashboard live, GreenMeter counting
P5 · Hardware & Hardening	5–6	M9	2–3 sensor slots + servo gate; validation & rate limits
P6 · Test, Deploy, Rehearse	5.5–6.5	—	M4: deployed on public URL; demo rehearsed twice

PARALLELISE LIKE A TEAM OF FOUR

Backend and frontend of each phase in parallel (2 people); one person owns the digital twin + analytics while another owns QR + edge. The hardware sprint (P5) is one person + one soldering afternoon. Demo every Friday to your guide — the milestone diamonds on the Gantt are those demos.

11 Testing Checklist — 10 Must-Pass Scenarios

- [] **Roles:** register/login for all three roles; wrong password and wrong role correctly rejected.
- [] **Double-booking:** two browsers submit overlapping bookings for the same slot — exactly one succeeds.
- [] **Availability math:** counts stay correct after every state change (book, cancel, expire, check-in, check-out).
- [] **QR forgery:** an edited/re-signed QR and a raw fake ID are both rejected at the kiosk.
- [] **QR expiry:** a screenshot of yesterday's QR is rejected (expired exp claim).
- [] **Replay:** a second check-in attempt with the same used token is refused.
- [] **No-show:** a booking that never checks in auto-expires after 20 minutes and frees the slot.
- [] **Check-out:** slot returns to AVAILABLE instantly; session shows correct duration in history.
- [] **Overstay:** session exceeding the booked window is flagged and alerts the admin; force-end works.
- [] **Geofence:** leaving the campus boundary auto-closes the session after the 5-minute grace period.

SEED DATA IS A TEST ASSET

Generate 6–12 months of synthetic reservations and sessions before demo day — your forecast heatmap, peak-hour analytics and GreenMeter all need history to look alive.

12 Demo Script & Viva Preparation

The 2-minute demo (rehearse it — timing is everything)

Admin (30 s): log in, tweak a lot, show the live grid and the analytics dashboard.

Student — forecast (20 s): open the digital twin map; show P1 nearly full and the heatmap saying “P1 full by 9:40”.

Student — booking (20 s): book 10:00–12:00; ClassSync suggests the lot nearest the 10 AM class; confirm.

QR arrives (10 s): the pass appears in-app and the email lands — show both screens.

Check-in (15 s): a second phone (kiosk) scans the QR → green “checked in” → on the projector, the slot flips red live.

Attack demo (10 s): scan yesterday's screenshot → red REJECTED. Say: “signature plus expiry” and move on.

Check-out + close (15 s): exit scan → receipt with duration and CO₂ saved → end on the GreenMeter counter.

The 8 questions evaluators will actually ask

Q. How do you prevent two users booking the same slot?

A. A DB transaction with a row lock (SELECT ... FOR UPDATE) plus a re-check for overlaps before insert; PostgreSQL's EXCLUDE constraint is belt-and-braces on top.

Q. Why a signed JWT inside the QR instead of just the booking ID?

A. A raw ID is trivially forgeable. The signature proves the server issued it; the exp claim bounds it in time; server-side single-use state kills replays.

Q. What if the campus internet is down at the gate?

A. The EdgeGuard kiosk verifies the HMAC signature and time window locally, caches a revocation list and queues events to sync when the link returns.

Q. How does geofence auto check-out work?

A. The PWA reports location; haversine distance to the campus boundary; on exit plus a 5-minute grace period the session closes and the user is notified.

Q. Where does the ML model's training data come from?

A. The reservations and sessions tables — seeded with synthetic history for the demo, then self-improving from real usage.

Q. What happens when a user no-shows?

A. A scheduled job expires the reservation 20 minutes after the start time, frees the slot and emails the user.

Q. How would you scale to five campuses?

A. The API is stateless behind a load balancer; campuses are rows, not codebases; MQTT fans out sensor traffic; read replicas serve availability queries.

Q. Why not cameras (ANPR) everywhere instead of QR?

A. Cost and privacy. QR needs a phone and a printer; ANPR is an optional express layer for staff vehicles — the architecture accepts both.

13 Future Scope — The Final Slide

ANPR-first entry — camera-only entry with QR as fallback, not the reverse.

Payments & fines — Razorpay/Stripe for penalties and premium slots; the FairFlow points system is the bridge.

Native app + push — the PWA graduates to iOS/Android with background geofencing.

Multi-campus — one control plane, many campuses, per-campus branding.

Smart navigation — turn-by-turn guidance to the reserved slot, then indoor positioning.

EV integration — reserve the charger, not just the slot; billing by kWh.

THE CLOSING LINE FOR YOUR PRESENTATION

“We didn't just digitise parking — we predicted it. ParkIQ turns every slot on campus into a live, intelligent, self-managing resource.”

Core first, wow second: build Modules 1–6 solid, then layer the S.M.A.R.T. features on top. That is how you win both the demo and the viva.